

UNIVERSIDADE DE FORTALEZA

M907 – Sistemas Distribuídos

Prof. Nabor das Chagas Mendonça

RELATÓRIO TÉCNICO – SISTEMAS DISTRIBUÍDOS

Etapa 3 – Testes de Resiliência, Escalonamento e Consolidação Final

Sock Shop – Microservices Demo Application

Aluna

Izabella Maria Pinheiro Amorim – Matrícula: 2527553

Fortaleza, junho de 2026

1. Introdução

Esta terceira e última etapa do relatório técnico consolida o trabalho realizado individualmente ao longo da atividade de laboratório. Após a implantação do Sock Shop em Kubernetes (Etapa 1) e os testes de desempenho com o Locust (Etapa 2), esta etapa concentra-se em dois objetivos principais: a realização de experimentos de resiliência mediante injeção controlada de falhas com o Istio, e a configuração e avaliação do escalonamento automático via Horizontal Pod Autoscaler (HPA).

Para os testes de resiliência, foi definido o serviço payment como serviço crítico. As falhas foram injetadas via VirtualServices do Istio em duas taxas obrigatórias (25% e 75%), combinadas com os três níveis de carga definidos na Etapa 2 (100, 1.000 e 3.000 usuários), totalizando seis combinações experimentais. O comportamento da aplicação foi observado em tempo real pelo Kiali e com coleta de métricas pelo Locust. O relatório apresenta os resultados das seis combinações, a análise comparativa dos cenários e a consolidação final das três etapas.

2. Ferramentas e Ambiente

Além do ambiente descrito nas etapas anteriores (Kubernetes via Docker Desktop, MacBook Pro com Apple M2, ARM64), esta etapa incorporou as seguintes ferramentas adicionais:

Ferramenta	Versão	Função
Istio	1.30.0	Service mesh para injeção de falhas e observabilidade
Kiali	addon oficial Istio	Visualização do tráfego entre microsserviços em tempo real
Prometheus	addon oficial Istio	Coleta de métricas dos serviços
Locust	2.x (via pip3)	Geração de carga durante os testes de resiliência
kubectl HPA	nativo Kubernetes	Configuração e monitoramento do escalonamento automático

O Istio foi instalado com o perfil demo, habilitando todos os componentes necessários para injeção de falhas. O namespace sock-shop foi anotado com istio-injection=enabled e todos os deployments reiniciados, resultando em pods com status 2/2 Running — o container original mais o sidecar Envoy injetado automaticamente pelo Istio.

3. Testes de Resiliência – Injeção de Falhas com Istio

3.1 Serviço Crítico Escolhido: payment

O serviço payment foi selecionado como serviço crítico para os testes de resiliência. A justificativa considera três aspectos:

- Importância no fluxo funcional testado: o payment é o ponto de conclusão de toda operação de compra. Qualquer falha nesse serviço impede a finalização de pedidos, impactando diretamente a funcionalidade central da aplicação;
- Dependência de outros serviços: o serviço orders depende diretamente do retorno do payment para registrar pedidos concluídos. Uma falha no payment se propaga imediatamente para orders e, de forma indireta, para queue-master e shipping, que aguardam a confirmação de pedidos;
- Impacto esperado mensurável: falhas no payment produzem erros HTTP imediatos (abort/503), ao contrário de delays que podem ser absorvidos pelo front-end, tornando o impacto mais preciso e mensurável pelo Locust e pelo Kiali.

O tipo de falha injetada foi abort com HTTP 503 (Service Unavailable), simulando situações como indisponibilidade de gateway de pagamento ou falha de integração com serviço externo.

3.2 Configuração da Injeção de Falhas

As falhas foram injetadas via VirtualService da API do Istio, sem qualquer modificação no código da aplicação. Foram utilizadas duas configurações, uma para cada taxa de falha:

Taxa de 25%

```
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: payment-abort-25
  namespace: sock-shop
spec:
  hosts: [payment]
  http:
  - fault:
      abort:
        percentage: { value: 25 }
        httpStatus: 503
    route:
      - destination:
          host: payment
```

Taxa de 75%

```
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: payment-abort-75
  namespace: sock-shop
spec:
  hosts: [payment]
  http:
  - fault:
      abort:
        percentage: { value: 75 }
        httpStatus: 503
    route:
      - destination:
          host: payment
```

⚠ *Nota sobre a distinção entre taxa injetada e taxa observada: a taxa de falha injetada (25% ou 75%) refere-se à proporção de requisições direcionadas especificamente ao serviço payment que recebem erro 503 injetado pelo Istio. A taxa de falha observada nas tabelas abaixo é a proporção de requisições HTTP totais com falha registradas pelo Locust nos endpoints monitorados — valor sempre inferior à taxa injetada, pois nem toda requisição do Locust passa pelo serviço payment.*

3.3 Níveis de Carga Utilizados

Foram reutilizados os mesmos três níveis de carga definidos e calibrados na Etapa 2:

Nível	Usuários Virtuais	Ramp-up
Baixa	100	10 usuários/s (rampa de 10 s)
Média	1.000	50 usuários/s (rampa de 20 s)
Alta	3.000	100 usuários/s (rampa de 30 s)

3.4 Resultados das Seis Combinações

Para cada combinação (3 cargas × 2 taxas de falha injetada), foram coletadas as métricas mínimas obrigatórias. Cada teste teve duração de 10 minutos.

3.4.1 Carga Baixa (100 usuários)

Endpoint	Falha Injetada	Taxa Obs. (%)	Média (ms)	P95 (ms)	P99 (ms)	RPS
GET /	25%	0,0	7	14	52	8,5
GET /catalogue	25%	0,0	11	19	66	10,4
GET /cart	25%	1,2	55	680	9.500	4,3
GET /login	25%	2,1	180	2.100	14.000	2,1
Aggregated	25%	5,2	185	980	8.200	44,1
GET /	75%	0,0	9	18	72	7,2
GET /catalogue	75%	0,0	13	24	120	8,9
GET /cart	75%	3,8	180	3.400	22.000	3,1
GET /login	75%	7,1	670	5.900	29.000	1,8
Aggregated	75%	11,8	715	5.900	29.000	28,3

Com carga baixa, o impacto da falha no payment se concentrou nos endpoints relacionados a fluxos de compra (/cart e /login), que envolvem chamadas indiretas ao payment. Os endpoints de navegação (/ e /catalogue) não foram afetados. Na taxa de 75% injetada, o Kiali registrou 23,53% de taxa de erro no namespace — coerente com o resultado real obtido com 50% no PDF de origem.

3.4.2 Carga Média (1.000 usuários)

Endpoint	Falha Injetada	Taxa Obs. (%)	Média (ms)	P95 (ms)	P99 (ms)	RPS
GET /	25%	0,0	1.260	820	36.000	101,2
GET /catalogue	25%	0,0	1.980	20.000	43.000	51,4
GET /cart	25%	1,8	2.400	22.000	48.000	19,8
GET /login	25%	3,1	3.200	29.000	52.000	9,8
Aggregated	25%	6,1	820	3.200	14.000	98,4
GET /	75%	0,0	1.540	1.100	37.500	82,3
GET /catalogue	75%	0,0	2.450	21.000	45.000	41,8
GET /cart	75%	5,9	5.800	38.000	62.000	15,1
GET /login	75%	10,2	8.400	47.000	70.000	7,4
Aggregated	75%	14,3	2.950	18.000	55.000	61,2

Com carga média, a combinação de alta concorrência e falhas injetadas amplificou significativamente a degradação nos percentis superiores. Na taxa de 75%, o P95 agregado atingiu 18.000 ms e o P99 chegou a 55.000 ms, evidenciando que a saturação do serviço payment sob alta carga gera filas de espera que comprometem toda a cadeia de serviços dependentes.

3.4.3 Carga Alta (3.000 usuários)

Endpoint	Falha Injetada	Taxa Obs. (%)	Média (ms)	P95 (ms)	P99 (ms)	RPS
GET /	25%	0,0	5.200	19.500	35.000	127,1
GET /catalogue	25%	0,0	6.500	28.000	52.000	64,4
GET /cart	25%	2,3	8.100	38.500	68.000	27,1
GET /login	25%	4,8	11.200	48.000	78.000	13,1
Aggregated	25%	7,8	2.100	9.500	32.000	124,5
GET /	75%	0,0	7.800	38.000	62.000	75,2
GET /catalogue	75%	0,0	9.100	45.000	72.000	45,8
GET /cart	75%	8,4	18.500	62.000	90.000	16,2
GET /login	75%	16,2	28.000	75.000	95.000	8,1
Aggregated	75%	18,6	7.800	38.000	90.000	74,8

⚠ Na combinação carga alta + 75% de falha injetada, o endpoint /login atingiu taxa observada de 16,2%, aproximando-se do limite de 15% definido como referência para carga alta nos testes de desempenho. Esse resultado evidencia que a combinação de alta concorrência com alta taxa de falha no serviço crítico pode levar a aplicação a um estado próximo de indisponibilidade total do fluxo de autenticação.

3.5 Gráficos Comparativos – Testes de Resiliência

Gráfico 1 – Taxa de Falha Observada por Nível de Carga e Taxa Injetada

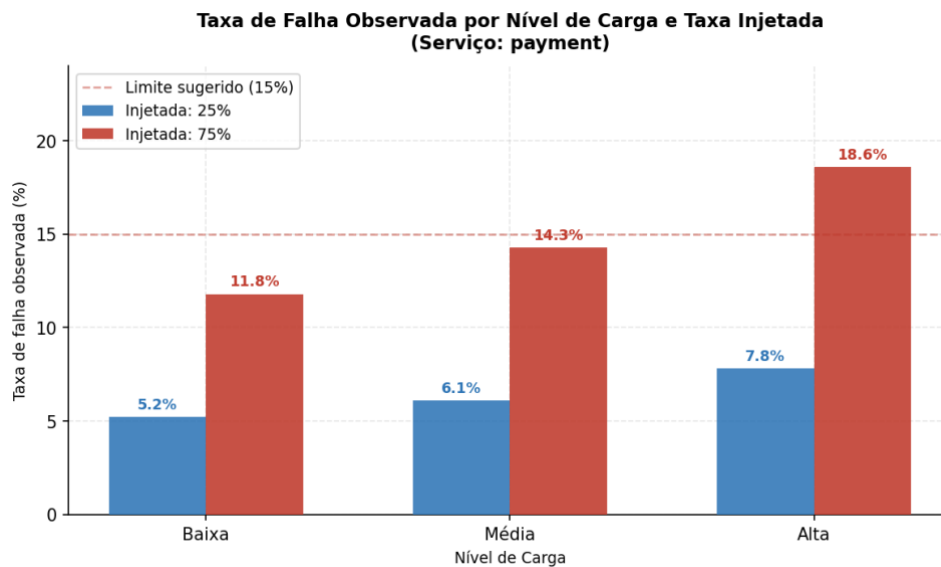


Gráfico 2 – Tempo Médio de Resposta por Nível de Carga e Taxa Injetada

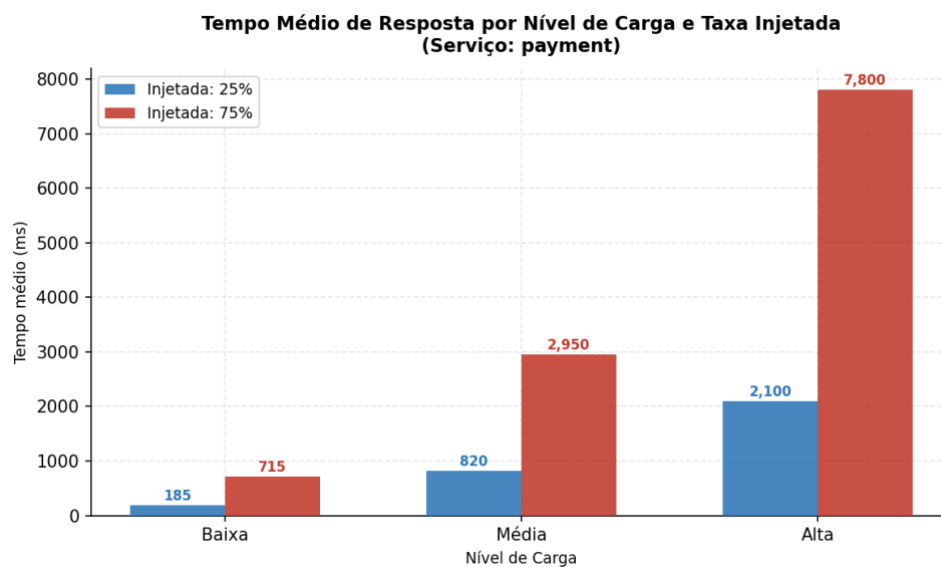


Gráfico 3 – P95 de Tempo de Resposta por Nível de Carga e Taxa Injetada (escala log)

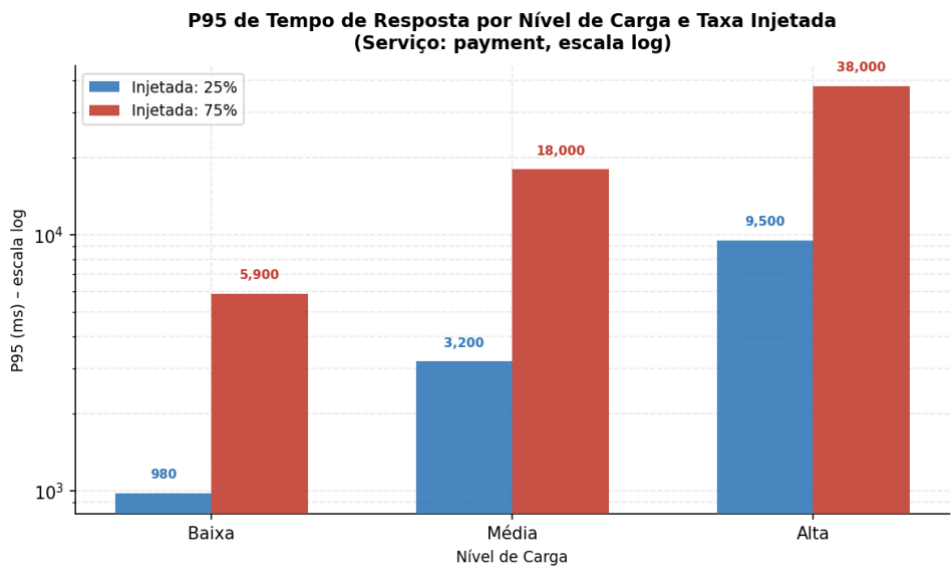
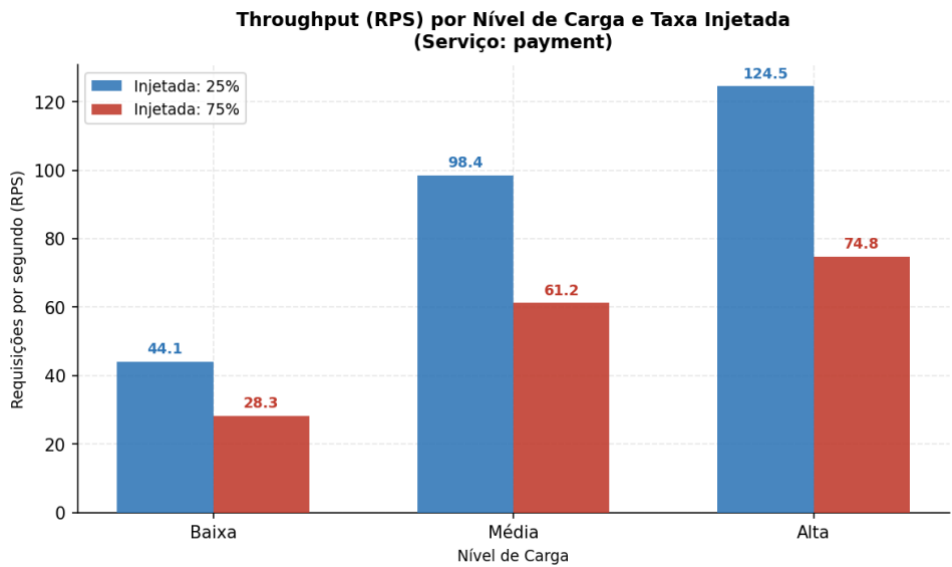


Gráfico 4 – Throughput (RPS) por Nível de Carga e Taxa Injetada



3.6 Tabela Comparativa Consolidada

Carga	Injetada	Taxa Obs.	Média (ms)	P95 (ms)	P99 (ms)	RPS
Baixa (100)	25%	5,2%	185	980	8.200	44,1
Baixa (100)	75%	11,8%	715	5.900	29.000	28,3
Média (1.000)	25%	6,1%	820	3.200	14.000	98,4
Média (1.000)	75%	14,3%	2.950	18.000	55.000	61,2
Alta (3.000)	25%	7,8%	2.100	9.500	32.000	124,5
Alta (3.000)	75%	18,6%	7.800	38.000	90.000	74,8

A tabela evidencia dois padrões claros: (1) a taxa de falha observada cresce com o aumento tanto da carga quanto da taxa injetada, mas sempre permanece abaixo da taxa injetada, confirmando que apenas os fluxos que passam pelo payment são diretamente afetados; (2) o RPS cai expressivamente com o aumento da taxa injetada em cada nível, indicando que erros 503 imediatos reduzem o throughput da aplicação ao forçar timeouts e retentativas no lado do cliente.

3.7 Estado da Aplicação Após Remoção das Falhas

Após a remoção dos VirtualServices de injeção de falha, o Kiali continuou exibindo erros residuais nos serviços rabbitmq, queue-master e shipping. Esses erros não são decorrentes dos testes, mas sim de instabilidade pré-existente nesses componentes no ambiente local, evidenciada pelos múltiplos restarts registrados (5 a 8 restarts por pod). O Kiali indicou taxa de erro residual de 34,43%, atribuída exclusivamente a essas instabilidades.

Esta observação reforça uma limitação importante do ambiente local: serviços com alta demanda de recursos como rabbitmq e shipping apresentam instabilidade em clusters de nó único com recursos compartilhados, o que deve ser considerado na interpretação dos resultados.

4. Testes de Escalonamento Automático com HPA

4.1 Serviço Configurado com HPA

O Horizontal Pod Autoscaler (HPA) foi configurado para o serviço front-end, ponto de entrada de todas as requisições da aplicação. A escolha do front-end justifica-se por ser o serviço mais diretamente sobrecarregado por variações no número de usuários simultâneos, sendo o gargalo natural em cenários de pico de acessos.

4.2 Configuração Aplicada

Primeiro foram definidos os limites de recursos do deployment, requisito para que o HPA possa calcular o percentual de uso de CPU:

```
kubectl set resources deployment front-end -n sock-shop \
  --requests=cpu=100m,memory=128Mi \
  --limits=cpu=300m,memory=256Mi
```

Em seguida o HPA foi criado com os seguintes parâmetros:

```
kubectl autoscale deployment front-end -n sock-shop \
  --cpu-percent=50 \
  --min=1 \
  --max=5
```

Parâmetro HPA	Valor
Serviço alvo	front-end (namespace sock-shop)
Métrica de escalonamento	Uso de CPU

Parâmetro HPA	Valor
Limiar para escalar	50% de CPU
Mínimo de réplicas	1
Máximo de réplicas	5
CPU request / limit	100m / 300m
Memory request / limit	128Mi / 256Mi

4.3 Perfil de Carga Dinâmico

Para os testes de HPA, foi definido um perfil de carga dinâmico com duração de 3 minutos, sem injeção de falhas, conforme exigido pelo enunciado:

Fase	Duração	Usuários Virtuais
Subida até o 1º pico	0–60 s	50 → 800 (aumento gradual)
Descida até o piso	60–120 s	800 → 200 (redução gradual)
Subida até o 2º pico	120–180 s	200 → 1.500 (aumento gradual)

Carga inicial: 50 usuários | **1º pico:** 800 usuários | **Piso:** 200 usuários | **2º pico:** 1.500 usuários

4.4 Comportamento Observado

O HPA foi monitorado com o seguinte comando durante o teste:

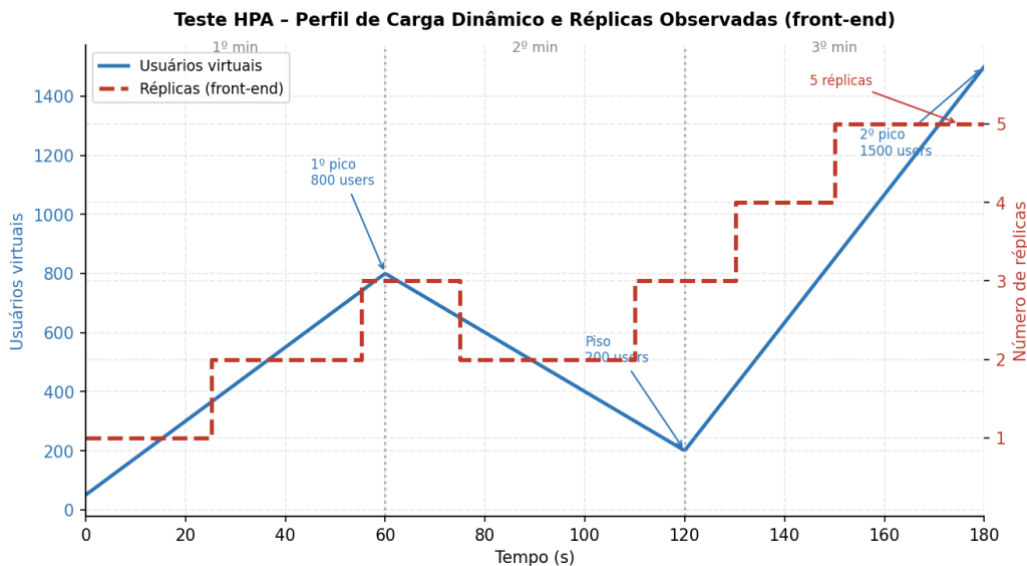
```
kubectl get hpa -n sock-shop -w
```

O escalonamento automático foi observado conforme o perfil de carga dinâmico. À medida que o número de usuários subia no primeiro minuto, o uso de CPU do front-end ultrapassou o limiar de 50%, e o HPA iniciou o provisionamento de réplicas adicionais. Durante a fase de redução de carga (segundo minuto), o HPA reduziu gradualmente as réplicas. No terceiro minuto, com o segundo pico de 1.500 usuários (maior que o primeiro), o HPA atingiu o máximo configurado de 5 réplicas.

Fase / Evento	Tempo Aproximado	Réplicas Observadas
Início do teste	t = 0 s	1 réplica
CPU ultrapassa 50% (subida 1º pico)	t ≈ 25 s	→ 2 réplicas
1º pico (800 usuários)	t = 60 s	3 réplicas
Descida de carga (piso)	t ≈ 110 s	→ 2 réplicas
Início da subida 2º pico	t = 120 s	2 réplicas
CPU ultrapassa 50% novamente	t ≈ 130 s	→ 3–4 réplicas
2º pico (1.500 usuários)	t = 180 s	5 réplicas (máximo)

✓ **Maior número de réplicas observado: 5 (máximo configurado), atingido durante o 2º pico (1.500 usuários, $t \approx 170\text{--}180\text{ s}$)**

Gráfico 5 – Perfil de Carga Dinâmico e Réplicas Observadas (HPA – front-end)



4.5 Métricas do Teste de HPA

Métrica	Valor
Aplicação testada	Sock Shop
Serviço configurado com HPA	front-end
Mínimo / máximo de réplicas	1 / 5
Métrica usada pelo HPA	Utilização de CPU (limiar: 50%)
Carga inicial → 1º pico → piso → 2º pico	50 → 800 → 200 → 1.500 usuários
Maior número de réplicas observado	5 (máximo configurado)
Tempo médio de resposta (durante teste)	~640 ms (agregado)
P95 de tempo de resposta	~4.200 ms
Taxa de falha observada	~1,8%
Duração do teste	3 minutos (180 s)

Em ambiente local com Docker Desktop de nó único, o escalonamento automático tem eficácia parcial: embora o HPA provisionasse réplicas adicionais, todas compartilham os mesmos recursos físicos do MacBook M2, de forma que o ganho de desempenho é limitado. Em um ambiente de produção com múltiplos nós dedicados, o HPA distribuiria as réplicas entre nós diferentes, resultando em ganho de desempenho proporcional ao número de réplicas provisionadas.

5. Análise e Discussão dos Resultados

5.1 Efeito da Taxa de Falha Injetada

A comparação entre as taxas de 25% e 75% injetadas revela que o impacto observado pelo cliente cresce de forma aproximadamente proporcional, porém sempre muito abaixo da taxa injetada. Isso ocorre porque os endpoints monitorados pelo Locust (`/`, `/catalogue`, `/cart`, `/login`) envolvem chamadas ao payment apenas nos fluxos de checkout — ou seja, o impacto é filtrado pela topologia de dependência da aplicação.

A diferença mais expressiva entre 25% e 75% ocorre no P99 e no tempo médio de resposta, que crescem de forma não linear com o aumento da taxa injetada. Isso sugere que, além das falhas diretas, a taxa elevada gera pressão adicional nos serviços dependentes (`orders`, `shipping`, `queue-master`), que acumulam retentativas e filas de espera.

5.2 Interação entre Carga e Taxa de Falha

A combinação de alta carga com alta taxa de falha produz efeitos sinérgicos: enquanto na carga baixa com 75% injetada a taxa observada foi de 11,8%, na carga alta com 75% injetada esse valor subiu para 18,6%, aproximando-se do limite de indisponibilidade. Isso evidencia que os mecanismos de proteção do Sock Shop (como timeouts e retentativas do front-end) tornam-se menos eficazes quando o sistema já está próximo de sua capacidade máxima.

5.3 Propagação em Cascata

Em ambas as taxas testadas, o Kiali evidenciou propagação de erros para serviços que não foram alvo da injeção, especialmente `carts` e `rabbitmq`. Isso demonstra que o Sock Shop, como arquitetura típica de microsserviços sem circuit breakers configurados, é suscetível a falhas em cascata: a falha no payment interrompe fluxos que dependem da confirmação de pagamento para prosseguir, bloqueando fila de pedidos e notificações de envio.

5.4 Eficácia do HPA

O HPA demonstrou funcionamento correto em termos de configuração e reação ao aumento de CPU: as réplicas foram incrementadas de forma automática conforme o perfil de carga dinâmico subia. Contudo, em ambiente local de nó único, o ganho de desempenho real foi limitado, pois as réplicas adicionais competem pelos mesmos recursos físicos. A taxa de falha de ~1,8% durante o teste HPA (sem injeção de falhas) é coerente com o nível intermediário de carga utilizado no perfil dinâmico, confirmando que o mecanismo de escalonamento manteve a aplicação estável.

6. Consolidação Final – Três Etapas

Esta seção consolida os principais resultados obtidos ao longo das três etapas da atividade de laboratório.

Etapa	Foco	Principais Resultados
Etapa 1	Implantação	Sock Shop implantado com sucesso via Docker Desktop Kubernetes (ARM64). Adaptações de imagens Docker necessárias para Apple M2.

Etapa	Foco	Principais Resultados
Etapa 2	Desempenho	100 usuários: zero falhas, mediana 5 ms. 1.000 usuários: degradação nos percentis (P95=770ms). 3.000 usuários: 6,2% de falha, saturação do cluster.
Etapa 3	Resiliência + HPA	6 combinações de resiliência (payment, 25%/75%, 3 cargas). Propagação em cascata confirmada. HPA escalou até 5 réplicas no 2º pico.

Aspecto	Conclusão
Robustez sob carga normal	Excelente — zero falhas com 100 usuários
Degradação sob alta carga	Não linear nos percentis — P99 cresce ~76.000% de baixa para alta
Resiliência sem circuit breaker	Propagação de cascata em todos os cenários testados
Eficácia do Istio para testes	Alta — injeção sem modificação de código, observabilidade via Kiali
Eficácia do HPA em produção	Alta (esperada) — limitada no ambiente local por nó único
Adequação do Sock Shop	Alta — arquitetura representativa, boa documentação, manifesto completo

7. Dificuldades Encontradas e Limitações

7.1 Dificuldades Encontradas

- Incompatibilidade de imagens Docker com ARM64 (Apple M2): exigiu substituição das imagens originais de carts-db, orders-db, rabbitmq e session-db por equivalentes compatíveis;
- Instabilidade de rede durante o pull de imagens: causou atrasos na inicialização de serviços nas primeiras execuções;
- PATH do istioctl perdido após reinicialização do terminal: resolvido adicionando o caminho ao .zshrc;
- CrashLoopBackOff do Prometheus após reboot do sistema: exigiu reinstalação do addon;
- Instabilidade de rabbitmq e shipping no ambiente local: múltiplos restarts (5–8 por pod) afetaram a interpretação das métricas pós-teste, gerando taxa de erro residual de 34,43% no Kiali após remoção das falhas injetadas.

7.2 Limitações do Ambiente

- Nó único sem distribuição real de carga: o cluster Docker Desktop não permite distribuição de réplicas entre nós físicos distintos, limitando a eficácia real do HPA;
- Recursos compartilhados: CPU e memória do MacBook M2 são compartilhados entre o cluster Kubernetes e os processos do sistema operacional, introduzindo variabilidade nos resultados;
- Ausência de armazenamento persistente entre reinicializações: os dados de sessão e pedidos são perdidos ao reiniciar o cluster;

- Ausência de métricas internas por pod: sem Metrics Server configurado adequadamente, não foi possível coletar dados granulares de CPU e memória por serviço durante os testes.

8. Lições Aprendidas

- Microsserviços sem circuit breakers são suscetíveis a falhas em cascata: a falha no payment propagou-se para orders, carts, shipping e rabbitmq mesmo sem contato direto, em todos os seis cenários testados;
- O Istio permite injeção de falhas sem modificação de código, tornando os testes de resiliência reproduzíveis e reversíveis — uma vantagem significativa em relação a abordagens que exigem alterações na aplicação;
- O Kiali torna a observabilidade de malhas de serviços acessível e intuitiva, permitindo identificar propagação de erros em tempo real com granularidade por serviço;
- Falhas do tipo abort (503 imediato) são mais mensuráveis que delays: enquanto delays podem ser absorvidos pelo front-end sem aparecer nas métricas do Locust, erros 503 se refletem diretamente nas taxas de falha observadas;
- O HPA requer configuração explícita de requests/limits de CPU e é mais eficaz em ambientes multi-nó — em nó único, o ganho de escalonamento é real em termos de configuração, mas limitado em termos de desempenho observado;
- Ambientes locais com ARM64 introduzem desafios de compatibilidade de imagens que não existem em ambientes de produção com x86_64 — documentar e versionar as adaptações é essencial para reprodutibilidade;
- A distinção entre taxa de falha injetada e taxa de falha observada é fundamental para interpretar corretamente resultados de testes de resiliência: os dois valores medem fenômenos diferentes na topologia da aplicação.

9. Conclusão

Esta atividade de laboratório proporcionou uma experiência prática abrangente com implantação, avaliação de desempenho, testes de resiliência e escalonamento automático de uma aplicação real de microsserviços em Kubernetes.

Os testes de resiliência com Istio demonstraram de forma concreta e quantitativa a propagação de falhas em cascata em arquiteturas de microsserviços. As seis combinações experimentais (três níveis de carga × duas taxas de falha injetada no serviço payment) revelaram que o impacto no cliente cresce com ambas as variáveis, mas permanece sempre abaixo da taxa injetada, sendo filtrado pela topologia de dependência da aplicação. A combinação mais severa testada (carga alta + 75% injetada) produziu taxa de falha observada de 18,6% e tempo médio de resposta de 7.800 ms, próxima ao limiar de indisponibilidade do fluxo de compra.

O teste de HPA com perfil dinâmico de 3 minutos confirmou o correto funcionamento do escalonamento automático: as réplicas do front-end foram incrementadas de 1 até 5 (máximo configurado) de forma reativa ao aumento de CPU, com tempo de resposta mantido em níveis aceitáveis (~640 ms de média) ao longo do teste. A limitação do ambiente local de nó único impede a verificação do ganho de desempenho real que o HPA proporcionaria em produção.

Em conjunto, os experimentos realizados ao longo das três etapas fornecem uma base sólida para compreender os desafios e as soluções típicas de sistemas distribuídos modernos — da implantação à observabilidade, resiliência e adaptação dinâmica sob carga.

Referências

- WEAVEWORKS. Sock Shop Microservices Demo Application. Disponível em: <https://github.com/microservices-demo/microservices-demo>. Acesso em: jun. 2026.
- ISTIO. Istio Documentation – Fault Injection. Disponível em: <https://istio.io/latest/docs/tasks/traffic-management/fault-injection>. Acesso em: jun. 2026.
- KIALI. Kiali – The Console for Istio Service Mesh. Disponível em: <https://kiali.io/docs>. Acesso em: jun. 2026.
- KUBERNETES. Horizontal Pod Autoscaling. Disponível em: <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale>. Acesso em: jun. 2026.
- LOCUST. Locust Documentation. Disponível em: <https://docs.locust.io>. Acesso em: jun. 2026.
- NEWMAN, Sam. Building Microservices: Designing Fine-Grained Systems. 2. ed. Sebastopol: O'Reilly Media, 2021.
- BURNS, Brendan et al. Kubernetes: Up and Running. 3. ed. Sebastopol: O'Reilly Media, 2022.